Codificación de módulos Android

Dilan Giovanny Molina Noy

Centro de servicios financieros, Servicio Nacional de Aprendizaje

SENA

3118311: Análisis y desarrollo de software

Andrés Rubiano Cucarían

26 de marzo de 2026

I. Introducción

En esta evidencia se desarrolla la codificación de los módulos de una aplicación móvil Android para la gestión básica de una farmacia. La aplicación permite el inicio de sesión con roles, registro de usuarios, gestión de productos y gestión de citas, cumpliendo con los requerimientos definidos previamente en el análisis del sistema.

El desarrollo se realizó utilizando Android Studio, aplicando buenas prácticas de programación, arquitectura por capas y control de versiones.

II. Metodología de desarrollo

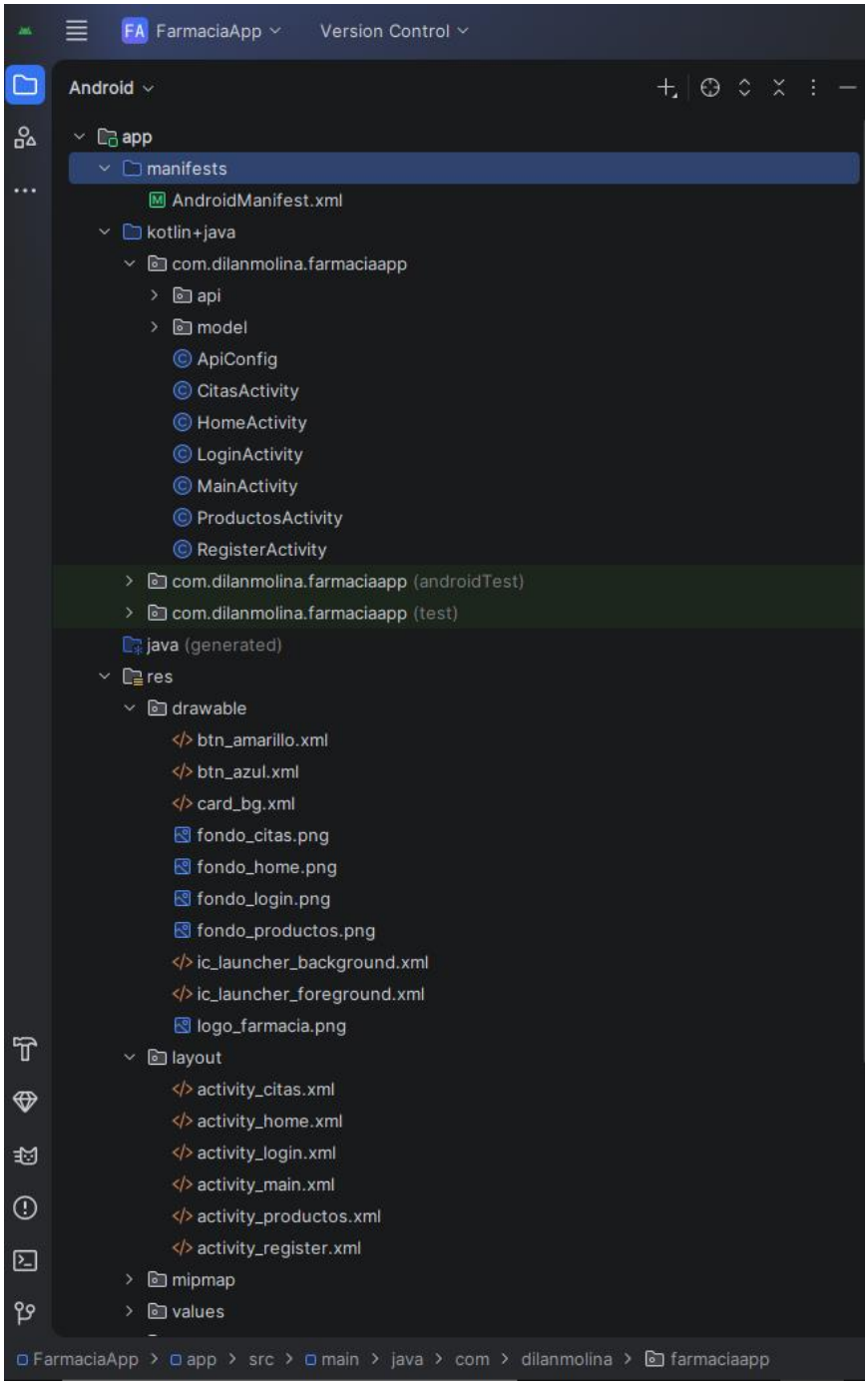
Se utilizó la metodología:

Desarrollo incremental

- Fases aplicadas:
- Análisis de requerimientos
- Diseño de interfaz
- Diseño arquitectura
- Codificación módulos
- Integración API
- Pruebas
- Documentación

III. Arquitectura del sistema

Para este desarrollo se implemento una arquitectura cliente-servidor en la que esta todo preparado para que se pueda consumir una API rest y posteriormente una base de datos, en la siguiente imagen podemos ver la estructura que tiene el proyecto en sus carpetas.



3.1.Capa presentación

- LoginActivity
- RegisterActivity
- HomeActivity
- ProductosActivity
- CitasActivity

3.2.Capa lógica

- Validación login
- Manejo roles
- Navegación

3.3.Capa datos

- ApiClient
- ApiService
- ApiConfig
- Modelos

IV. Funcionalidad

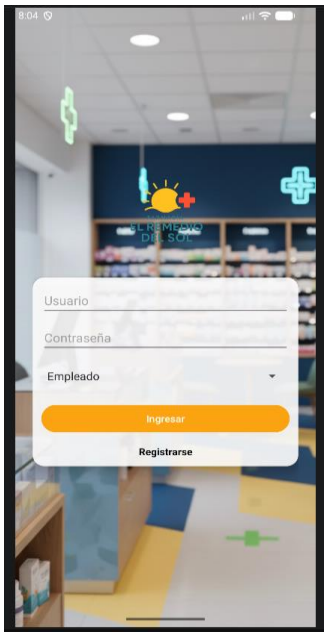
Clase LoginActivity

Responsabilidad: gestionar inicio de sesión

Ejemplo de código:

```java
LoginActivity.java

public class LoginActivity extends AppCompatActivity {

    2 usages
    EditText usuario, password;
    3 usages
    Spinner rol;
    2 usages
    Button login;
    2 usages
    TextView registrar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        usuario = findViewById(R.id.etUsuario);
        password = findViewById(R.id.etPassword);
        rol = findViewById(R.id.spRol);
        login = findViewById(R.id.btnLogin);
        registrar = findViewById(R.id.tvRegistrar);

        // Roles disponibles
        String[] roles = {"Empleado", "Cliente"};

        ArrayAdapter<String> adapter = new ArrayAdapter<>(
                context: this,
                android.R.layout.simple_spinner_dropdown_item,
                roles
        );

        rol.setAdapter(adapter);

        // BOTON LOGIN
        login.setOnClickListener( View v -> {

            String user = usuario.getText().toString();
            String pass = password.getText().toString();
            String tipo = rol.getSelectedItem().toString();
```

App ejecutada:



Clase RegisterActivity

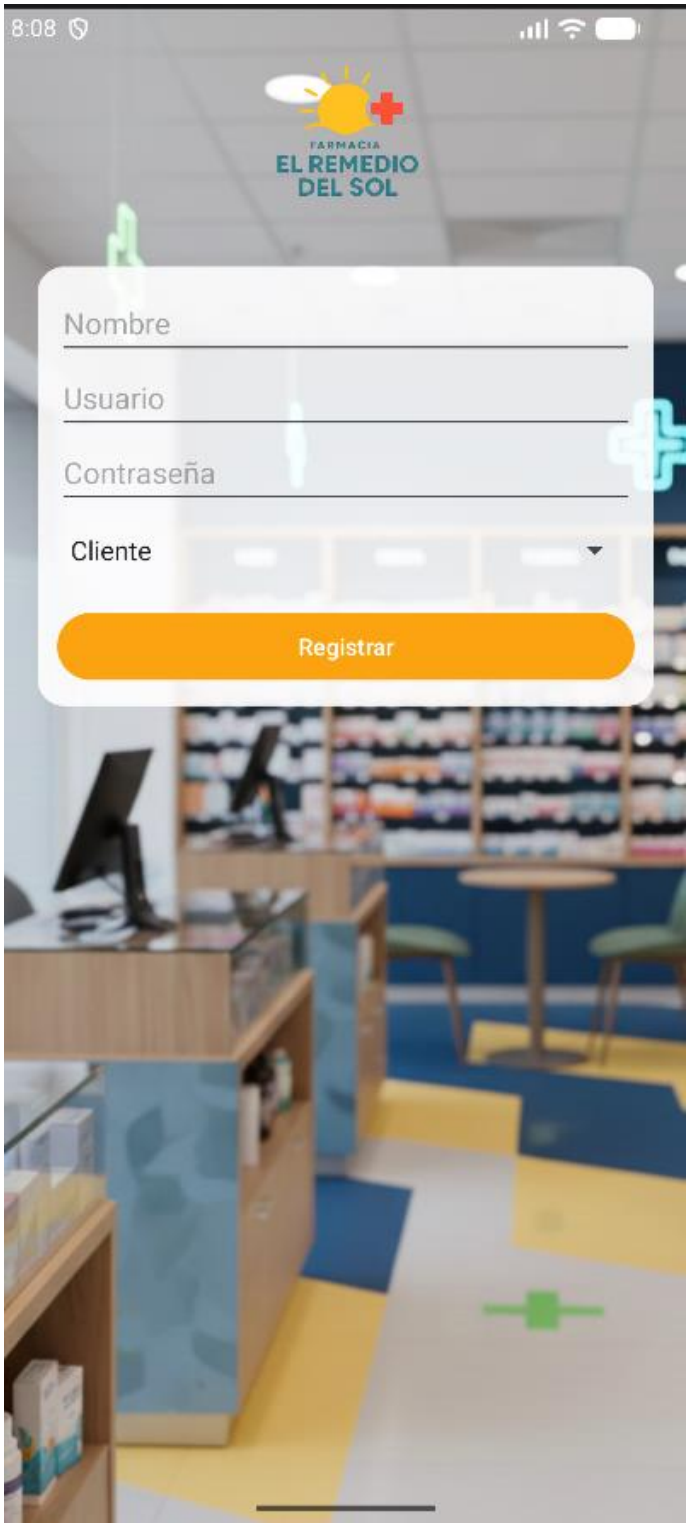
Responsabilidad: registrar usuarios

Ejemplo de código:

LoginActivity.java | RegisterActivity.java

```java
package com.dilanmolina.farmaciaapp;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.Spinner;

public class RegisterActivity extends AppCompatActivity {

    2 usages
    Spinner rol;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);

        rol = findViewById(R.id.spRol);

        String[] roles = {"Empleado","Cliente"};

        ArrayAdapter<String> adapter =
                new ArrayAdapter<>( context: this,
                        android.R.layout.simple_spinner_dropdown_item,
                        roles);

        rol.setAdapter(adapter);
    }
}
```

App ejecutada:



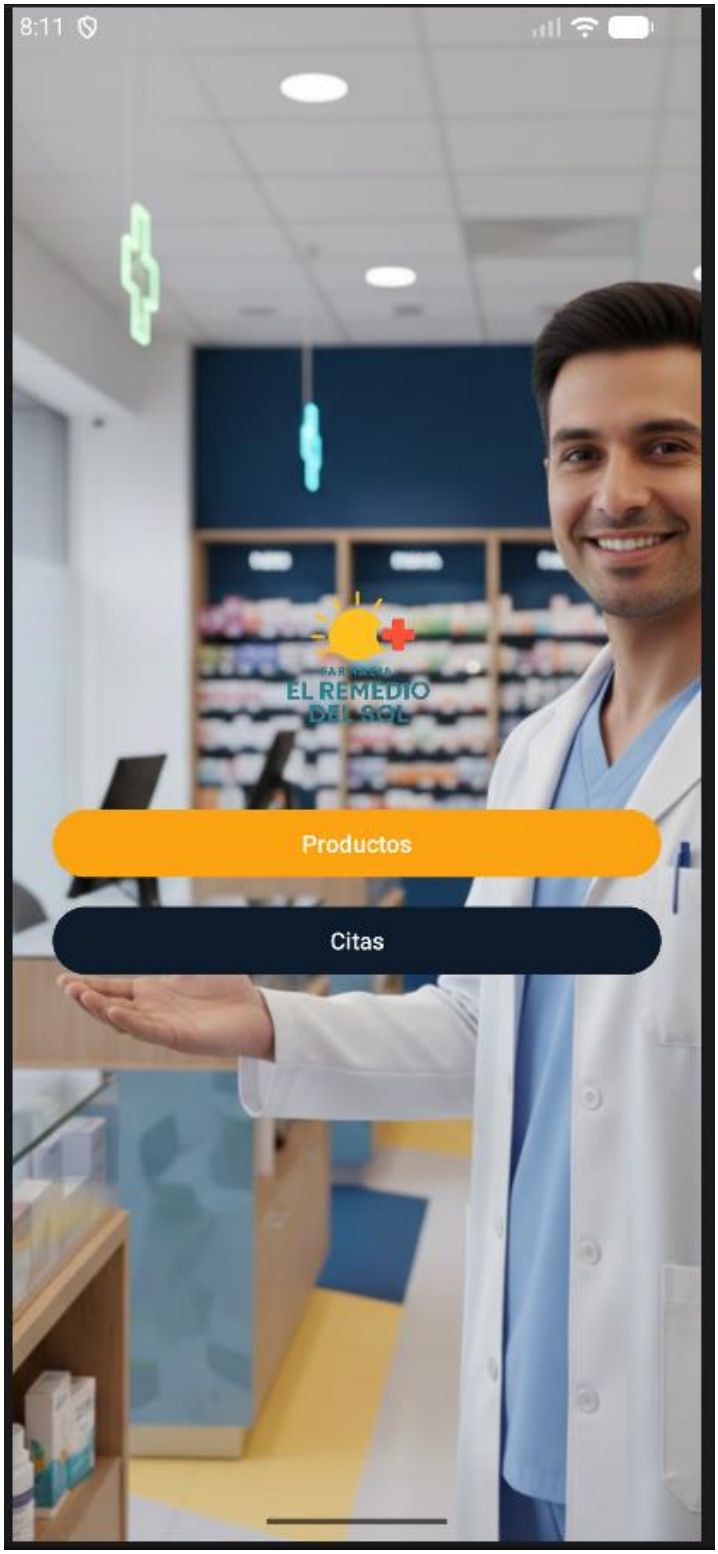
Clase HomeActivity

Responsabilidad: menú principal

Ejemplo de código:

```
LoginActivity.java   RegisterActivity.java   HomeActivity.java

package com.dilanmolina.farmaciaapp;

import ...

public class HomeActivity extends AppCompatActivity {

    2 usages
    Button productos, citas;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_home);

        productos = findViewById(R.id.btnProductos);
        citas = findViewById(R.id.btnCitas);

        productos.setOnClickListener( View v -> {
            startActivity(new Intent( packageContext: this, ProductosActivity.class));
        });

        citas.setOnClickListener( View v -> {
            startActivity(new Intent( packageContext: this, CitasActivity.class));
        });
    }
}
```

App ejecutada:



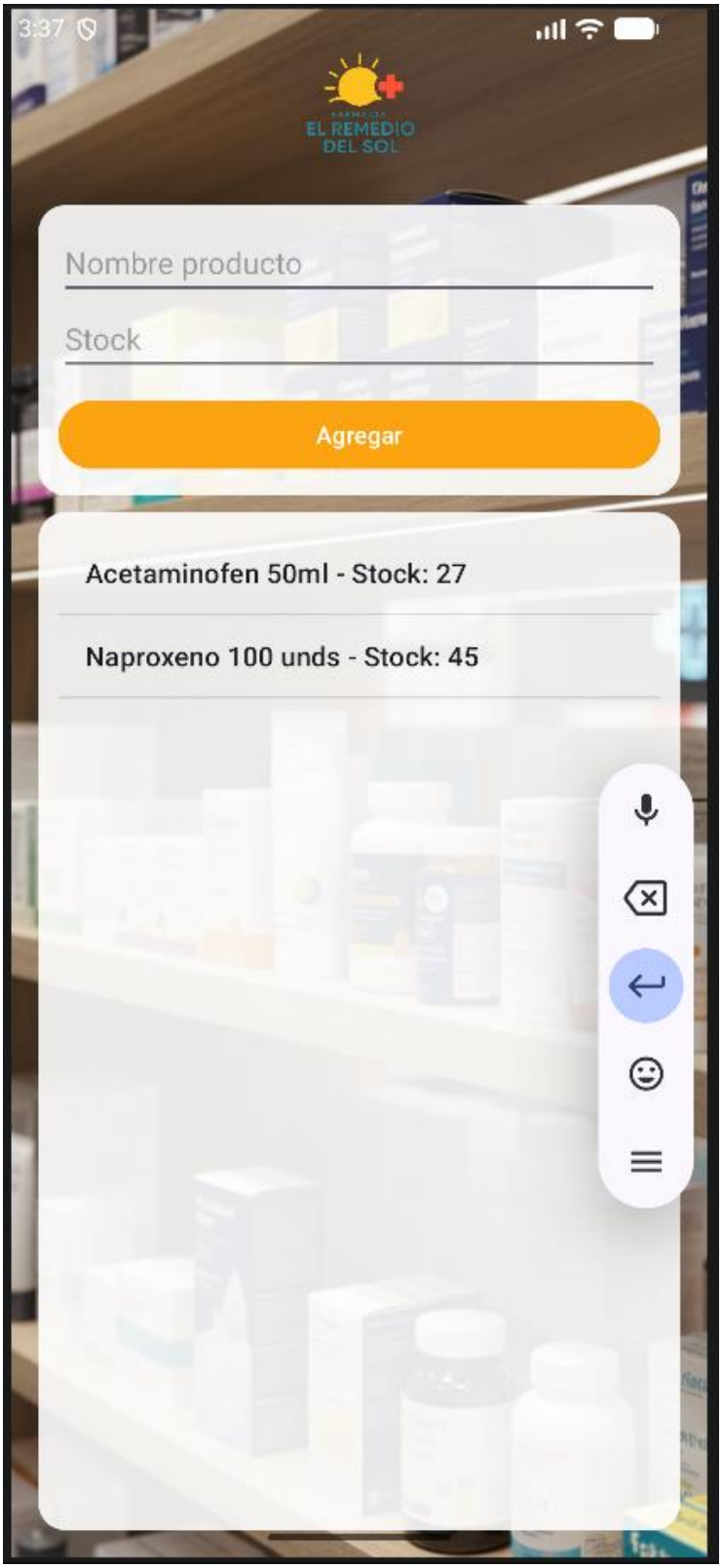
Clase ProductosActivity

Responsabilidad: CRUD productos

Ejemplo de código:

```
LoginActivity.java   RegisterActivity.java   HomeActivity.java   ProductosActivity.java

public class ProductosActivity extends AppCompatActivity {

    2 usages
    EditText nombre, stock;
    2 usages
    Button agregar;
    3 usages
    ListView lista;

    4 usages
    ArrayList<String> productos;
    4 usages
    ArrayAdapter<String> adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_productos);

        nombre = findViewById(R.id.etNombre);
        stock = findViewById(R.id.etStock);
        agregar = findViewById(R.id.btnAgregar);
        lista = findViewById(R.id.listProductos);

        productos = new ArrayList<>();

        adapter = new ArrayAdapter<>(
                context: this,
                android.R.layout.simple_list_item_1,
                productos
        );

        lista.setAdapter(adapter);

        agregar.setOnClickListener( View v -> {
            String n = nombre.getText().toString();
            String s = stock.getText().toString();

            productos.add(n + " - Stock: " + s);
            adapter.notifyDataSetChanged();
        });
```

App ejecutada:



Clase CitasActivity

Responsabilidad: CRUD citas

Ejemplo de código:

```
public class CitasActivity extends AppCompatActivity {

    2 usages
    EditText fecha, hora;
    2 usages
    Button agregar;
    3 usages
    ListView lista;

    4 usages
    ArrayList<String> citas;
    4 usages
    ArrayAdapter<String> adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_citas);

        fecha = findViewById(R.id.etFecha);
        hora = findViewById(R.id.etHora);
        agregar = findViewById(R.id.btnAgregarCita);
        lista = findViewById(R.id.listCitas);

        citas = new ArrayList<>();

        adapter = new ArrayAdapter<>(
                context: this,
                android.R.layout.simple_list_item_1,
                citas
        );

        lista.setAdapter(adapter);

        agregar.setOnClickListener( View v -> {
            String f = fecha.getText().toString();
            String h = hora.getText().toString();

            citas.add("Fecha: " + f + " Hora: " + h);
            adapter.notifyDataSetChanged();
        });
```

App ejecutada:

Clase ApiClient

Responsabilidad: conexión servidor

Ejemplo de código:

LoginActivity.java | RegisterActivity.java | HomeActivity.java | CitasActivity.java | ApiClient.java

```java
package com.dilanmolina.farmaciaapp.api;

import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

no usages
public class ApiClient {

    1 usage
    private static final String BASE_URL =
            "http://192.168.1.10:8080/api/";

    3 usages
    private static Retrofit retrofit;

    no usages
    public static Retrofit getClient(){

        if(retrofit == null){

            retrofit = new Retrofit.Builder()
                    .baseUrl(BASE_URL)
                    .addConverterFactory(
                            GsonConverterFactory.create()
                    )
                    .build();
        }

        return retrofit;
    }
}
```

Clase ApiService

Responsabilidad: endpoints REST

Ejemplo de código:

```
RegisterActivity.java   HomeActivity.java   CitasActivity.java   ApiClient.java   ApiService.java

package com.dilanmolina.farmaciaapp.api;

import ...

no usages
public interface ApiService {

    no usages
    @POST("login")
    Call<Usuario> login(@Body Usuario usuario);

    no usages
    @POST("register")
    Call<Usuario> register(@Body Usuario usuario);

}
```

V. Librerías utilizadas

- Retrofit
- Gson Converter
- OkHttp
- Material Design

Dependencias usadas:

- retrofit2
- gson

- okhttp
- material components

VI. Frameworks utilizados

- Android SDK
- Material Design Components
- Retrofit API Client

VII. Control de versiones

Se utilizó la herramienta de versiona miento Git como sistema de control de versiones.

Repositorio: https://github.com/dilanmolina4/ADSO-3118311-DILAN

VIII. Configuración de servidor y acceso a datos

La aplicación se conecta a un servidor mediante API REST.

Configuración:

ApiConfig.java

Contiene:

- URL servidor
- Endpoint API
- Configuración conexión

Ejemplo de código:

```
1   package com.dilanmolina.farmaciaapp;
2
    no usages
3   public class ApiConfig {
4
        no usages
5       public static final String BASE_URL =
6               "http://192.168.1.10:8080/api/";
7
8   }
```

Se utiliza Retrofit para consumir servicios REST.

Operaciones:

- Login usuario
- Registro usuario
- Obtener productos
- Crear productos
- Obtener citas
- Crear citas

IX. Ambientes de desarrollo

Los ambientes de desarrollo utilizados en este desarrollo fueron los siguientes:

- Android Studio
- Java
- SDK Android

- Emulador Android

X. Buenas prácticas aplicadas

Alguna de las buenas practicas que se realizaron durante este desarrollo fueron las siguientes:

- Separación por paquetes
- Arquitectura por capas
- Nombres descriptivos
- Componentes reutilizables
- Uso Retrofit
- Uso Modelos
- Uso Activities separadas
- Código modular

XI. Componentes reutilizables

- Botones personalizados
- Layouts reutilizables
- ApiClient reutilizable
- Modelos reutilizables
- Estilos globales

XII. Conclusión

Se desarrolló una aplicación móvil Android para la farmacia “El remedio del sol” la cual es funcional basada en los requerimientos del sistema. La solución implementa arquitectura cliente-servidor, separación por capas, uso de API REST y buenas prácticas de desarrollo. La aplicación permite la gestión de usuarios, productos y citas, cumpliendo con los criterios establecidos y asegurando escalabilidad y mantenimiento del sistema.